Celebal Excellence Internship 2026

Week 3 Assignment
Submitted By: Saksham Sharma

Objective

Use Subqueries, CTEs, and Window Functions to analyze sales data from the Superstore dataset. Apply advanced SQL techniques to solve real business queries including customer rankings, sales aggregations, and order-level analysis.

Database Schema

Created superstore_raw table to load the data:

Query-

```
CREATE TABLE superstore_raw (
   row_id       INT,
   order_id     VARCHAR(20),
   order_date   DATE,
   ship_date    DATE,
   ship_mode    VARCHAR(30),
   customer_id  VARCHAR(20),
   customer_name VARCHAR(50),
   segment      VARCHAR(20),
   country      VARCHAR(30),
   city         VARCHAR(50),
   state        VARCHAR(30),
   postal_code  INT,
   region       VARCHAR(20),
   product_id   VARCHAR(20),
   category     VARCHAR(30),
   sub_category VARCHAR(30),
   product_name VARCHAR(200),
   sales        DECIMAL(10,4),
   quantity     INT,
   discount     DECIMAL(5,2),
   profit       DECIMAL(10,4)
);
```

Output-

```
CREATE TABLE superstore_raw (    row_id      INT,    order_id    VARCHAR(20),    order_date   DATE,    ship_da...  0 row(s) affected
```

Step 1: Setup Data

Load superstore_raw

Query-

```
LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Sample - Superstore.csv'
INTO TABLE superstore_raw
CHARACTER SET latin1
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(
   row_id, order_id, @order_date, @ship_date, ship_mode,
   customer_id, customer_name, segment, country, city, state,
   postal_code, region, product_id, category, sub_category,
   product_name, sales, quantity, discount, profit
)
SET
   order_date = STR_TO_DATE(@order_date, '%m/%d/%Y'),
   ship_date  = STR_TO_DATE(@ship_date,  '%m/%d/%Y');
```

Output-

SELECT * FROM superstore_raw LIMIT 0, 10000	9994 row(s) returned

Create customers Table

Query-

```
CREATE TABLE customers (
   customer_id   VARCHAR(20),
   customer_name VARCHAR(50),
   segment       VARCHAR(20)
);

INSERT INTO customers
SELECT DISTINCT customer_id, customer_name, segment
FROM superstore_raw;
```

Output-

CREATE TABLE customers (customer_id VARCHAR(20), customer_name VARCHAR(50), segment V...	0 row(s) affected
INSERT INTO customers SELECT DISTINCT customer_id, customer_name, segment FROM superstore_raw	793 row(s) affected Records: 793 Duplicates: 0 Warnings: 0

Create products Table (SELECT DISTINCT)

Query-

```
CREATE TABLE products (
   product_id   VARCHAR(20),
   product_name VARCHAR(200),
   category     VARCHAR(50),
   sub_category VARCHAR(50)
);

INSERT INTO products
SELECT DISTINCT product_id, product_name, category, sub_category FROM superstore_raw;
```

Output-

CREATE TABLE products (product_id VARCHAR(20), product_name VARCHAR(200), category VARC...	0 row(s) affected
INSERT INTO products SELECT DISTINCT product_id, product_name, category, sub_category FROM superstore...	1894 row(s) affected Records: 1894 Duplicates: 0 Warnings: 0

Create orders Table (SELECT DISTINCT)

Query-

```
CREATE TABLE orders (
   order_id    VARCHAR(20),
   customer_id VARCHAR(20),
   product_id  VARCHAR(20),
   order_date  DATE,
   ship_date   DATE,
   ship_mode   VARCHAR(30),
   sales       DECIMAL(10,4),
   quantity    INT,
   discount    DECIMAL(5,2),
   profit      DECIMAL(10,4)
);

INSERT INTO orders
SELECT DISTINCT order_id, customer_id, product_id,
            order_date, ship_date, ship_mode,
            ROUND(sales, 4),
            quantity,
            ROUND(discount, 2),
            ROUND(profit, 4)
FROM superstore_raw;
```

Output-

CREATE TABLE orders (order_id VARCHAR(20), customer_id VARCHAR(20), product_id VARCHAR(20)...	0 row(s) affected		
INSERT INTO orders SELECT DISTINCT order_id, customer_id, product_id,	order_date, ship_date, ship...	9993 row(s) affected Records: 9993 Duplicates: 0 Warnings: 0	

Step 2: Perform Required Queries

Subquery 1 — Orders Where Sales > Average Sales

Query-

```
SELECT o.order_id, c.customer_name, p.product_name,
     ROUND(o.sales,2) AS sales
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
JOIN products  p ON o.product_id  = p.product_id
WHERE o.sales > (SELECT AVG(sales) FROM orders)
ORDER BY o.sales DESC
LIMIT 10;
```

Output-

order_id	customer_name	product_name	sales
CA-2014-145317	Sean Miller	Cisco TelePresence System EX90 Videoconferen...	22638.48
CA-2016-118689	Tamara Chand	Canon imageCLASS 2200 Advanced Copier	17499.95
CA-2017-140151	Raymond Buch	Canon imageCLASS 2200 Advanced Copier	13999.96
CA-2017-127180	Tom Ashbrook	Canon imageCLASS 2200 Advanced Copier	11199.97
CA-2017-166709	Hunter Lopez	Canon imageCLASS 2200 Advanced Copier	10499.97
CA-2016-117121	Adrian Barton	GBC Ibimaster 500 Manual ProClick Binding System	9892.74
CA-2014-116904	Sanjit Chand	Ibico EPK-21 Electric Binding System	9449.95
US-2016-107440	Bill Shonely	3D Systems Cube Printer, 2nd Generation, Mag...	9099.93
CA-2016-158841	Sanjit Engle	HP Designjet T520 Inkjet Large Format Printer -...	8749.95
CA-2016-143714	Christopher Conant	Canon imageCLASS 2200 Advanced Copier	8399.98

Subquery 2 — Highest Sales Order per Customer

Query-

```
SELECT c.customer_name, o.order_id,
     ROUND(o.sales,2) AS highest_sales
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE o.sales = (
   SELECT MAX(o2.sales)
   FROM orders o2
   WHERE o2.customer_id = o.customer_id
)
ORDER BY highest_sales DESC
LIMIT 10;
```

Output-

customer_name	order_id	highest_sales
Sean Miller	CA-2014-145317	22638.48
Tamara Chand	CA-2016-118689	17499.95
Raymond Buch	CA-2017-140151	13999.96
Tom Ashbrook	CA-2017-127180	11199.97
Hunter Lopez	CA-2017-166709	10499.97
Adrian Barton	CA-2016-117121	9892.74
Sanjit Chand	CA-2014-116904	9449.95
Bill Shonely	US-2016-107440	9099.93
Sanjit Engle	CA-2016-158841	8749.95
Christopher Conant	CA-2016-143714	8399.98

CTE 1 — Total Sales per Customer

Query-

```
WITH customer_sales AS (
   SELECT c.customer_id, c.customer_name,
        ROUND(SUM(o.sales),2) AS total_sales
   FROM orders o
   JOIN customers c ON o.customer_id = c.customer_id
   GROUP BY c.customer_id, c.customer_name
)
```

```
SELECT customer_id, customer_name, total_sales
FROM customer_sales ORDER BY total_sales DESC LIMIT 10;
```

Output-

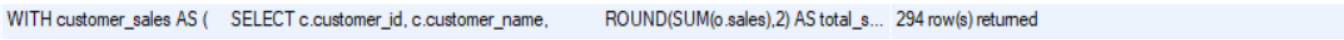
customer_id	customer_name	total_sales
SM-20320	Sean Miller	25043.05
TC-20980	Tamara Chand	19052.22
RB-19360	Raymond Buch	15117.34
TA-21385	Tom Ashbrook	14595.62
AB-10105	Adrian Barton	14473.57
KL-16645	Ken Lonsdale	14175.23
SC-20095	Sanjit Chand	14142.33
HL-15040	Hunter Lopez	12873.30
SE-20110	Sanjit Engle	12209.44
CC-12370	Christopher Conant	12129.07

CTE 2 — Customers Whose Total Sales Are Above Average (CTE + Subquery)

Query-

```
WITH customer_sales AS (
   SELECT c.customer_id, c.customer_name,
        ROUND(SUM(o.sales),2) AS total_sales
   FROM orders o
   JOIN customers c ON o.customer_id = c.customer_id
   GROUP BY c.customer_id, c.customer_name
)
SELECT customer_id, customer_name, total_sales
FROM customer_sales
WHERE total_sales > (SELECT AVG(total_sales) FROM customer_sales)
ORDER BY total_sales DESC;
```

Output-



Window Function 1 — Rank All Customers by Total Sales

Query-

```
WITH customer_sales AS (
   SELECT c.customer_id, c.customer_name,
        ROUND(SUM(o.sales),2) AS total_sales
   FROM orders o
   JOIN customers c ON o.customer_id = c.customer_id
   GROUP BY c.customer_id, c.customer_name
)
SELECT customer_name, total_sales,
     RANK() OVER (ORDER BY total_sales DESC) AS sales_rank
FROM customer_sales
LIMIT 10;
```

Output-

customer_name	total_sales	sales_rank
Sean Miller	25043.05	1
Tamara Chand	19052.22	2
Raymond Buch	15117.34	3
Tom Ashbrook	14595.62	4
Adrian Barton	14473.57	5
Ken Lonsdale	14175.23	6
Sanjit Chand	14142.33	7
Hunter Lopez	12873.30	8
Sanjit Engle	12209.44	9
Christopher Conant	12129.07	10

Window Function 2 — Row Number per Order Within Each Customer (PARTITION BY)

Query-

```
SELECT c.customer_name, o.order_id, o.order_date,
      ROUND(o.sales,2) AS sales,
      ROW_NUMBER() OVER (
         PARTITION BY o.customer_id
         ORDER BY o.order_date
      ) AS order_seq
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
ORDER BY c.customer_name, order_seq
LIMIT 12;
```

Output-

customer_name	order_id	order_date	sales	order_seq
Aaron Bergman	CA-2014-152905	2014-02-18	12.62	1
Aaron Bergman	CA-2014-156587	2014-03-07	17.94	2
Aaron Bergman	CA-2014-156587	2014-03-07	48.71	3
Aaron Bergman	CA-2014-156587	2014-03-07	242.94	4
Aaron Bergman	CA-2016-140935	2016-11-10	221.98	5
Aaron Bergman	CA-2016-140935	2016-11-10	341.96	6
Aaron Hawkins	CA-2014-122070	2014-04-22	9.91	1
Aaron Hawkins	CA-2014-122070	2014-04-22	247.84	2
Aaron Hawkins	CA-2014-113768	2014-05-13	8.00	3
Aaron Hawkins	CA-2014-113768	2014-05-13	279.46	4
Aaron Hawkins	US-2014-158400	2014-10-25	49.41	5
Aaron Hawkins	CA-2014-157644	2014-12-31	34.77	6

Window Function 3 — Top 3 Customers by Total Sales

Query-

```
WITH customer_sales AS (
   SELECT c.customer_id, c.customer_name,
        ROUND(SUM(o.sales),2) AS total_sales
   FROM orders o
   JOIN customers c ON o.customer_id = c.customer_id
   GROUP BY c.customer_id, c.customer_name
),
ranked AS (
```

```
  SELECT customer_name, total_sales,
      RANK() OVER (ORDER BY total_sales DESC) AS rnk
  FROM customer_sales
)
SELECT customer_name, total_sales, rnk
FROM ranked
WHERE rnk <= 3;
```

Output-

customer_name	total_sales	rnk
Sean Miller	25043.05	1
Tamara Chand	19052.22	2
Raymond Buch	15117.34	3

Step 3: Final Combined Query (JOIN + CTE + Window Function)

One single query showing Customer Name, Total Sales, and Rank:

Query-

```
WITH customer_sales AS (
  SELECT c.customer_id, c.customer_name, c.segment,
      ROUND(SUM(o.sales),2)      AS total_sales,
      COUNT(DISTINCT o.order_id)  AS total_orders
  FROM orders o
  JOIN customers c ON o.customer_id = c.customer_id
  GROUP BY c.customer_id, c.customer_name, c.segment
)
SELECT customer_name, segment, total_sales, total_orders,
    RANK() OVER (ORDER BY total_sales DESC) AS sales_rank
FROM customer_sales
ORDER BY sales_rank
LIMIT 15;
```

Output-

customer_name	segment	total_sales	total_orders	sales_rank
Sean Miller	Home Office	25043.05	5	1
Tamara Chand	Corporate	19052.22	5	2
Raymond Buch	Consumer	15117.34	6	3
Tom Ashbrook	Home Office	14595.62	4	4
Adrian Barton	Consumer	14473.57	10	5
Ken Lonsdale	Consumer	14175.23	12	6
Sanjit Chand	Consumer	14142.33	9	7
Hunter Lopez	Consumer	12873.30	6	8
Sanjit Engle	Consumer	12209.44	11	9
Christopher Conant	Consumer	12129.07	5	10
Todd Sumrall	Corporate	11891.75	6	11
Greg Tran	Consumer	11820.12	11	12
Becky Martin	Consumer	11789.63	4	13
Seth Vernon	Consumer	11470.95	10	14
Caroline Jumper	Consumer	11164.97	8	15

Mini Project: Customer Sales Insights

Q1. Who are the Top 5 Customers?

Query-

```
WITH customer_sales AS (
   SELECT c.customer_name,
        ROUND(SUM(o.sales),2) AS total_sales
   FROM orders o
   JOIN customers c ON o.customer_id = c.customer_id
   GROUP BY c.customer_id, c.customer_name
)
SELECT customer_name, total_sales,
      RANK() OVER (ORDER BY total_sales DESC) AS rnk
FROM customer_sales
ORDER BY rnk
LIMIT 5;
```

Output-

customer_name	total_sales	rnk
Sean Miller	25043.05	1
Tamara Chand	19052.22	2
Raymond Buch	15117.34	3
Tom Ashbrook	14595.62	4
Adrian Barton	14473.57	5

Q2. Who are the Bottom 5 Customers?

Query-

```
WITH customer_sales AS (
   SELECT c.customer_name,
        ROUND(SUM(o.sales),2) AS total_sales
   FROM orders o
   JOIN customers c ON o.customer_id = c.customer_id
   GROUP BY c.customer_id, c.customer_name
)
SELECT customer_name, total_sales,
      RANK() OVER (ORDER BY total_sales ASC) AS rnk
FROM customer_sales
ORDER BY rnk
LIMIT 5;
```

Output-

customer_name	total_sales	rnk
Thais Sissman	4.83	1
Lela Donovan	5.30	2
Carl Jackson	16.52	3
Mitch Gastineau	16.74	4
Roy Skaria	22.33	5

Q3. Which Customers Made Only One Order?

Query-

```
SELECT c.customer_name,
    COUNT(DISTINCT o.order_id) AS order_count
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.customer_name
HAVING COUNT(DISTINCT o.order_id) = 1
ORDER BY c.customer_name;
```

Output-

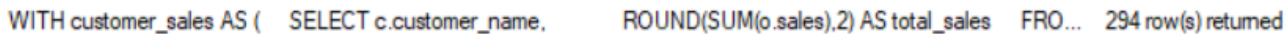
customer_name	order_count
Anemone Ratner	1
Anthony O'Donnell	1
Carl Jackson	1
Jenna Caffey	1
Jocasta Rupert	1
Lela Donovan	1
Mitch Gastineau	1
Patricia Hirasaki	1
Ricardo Emerson	1
Roland Murray	1
Susan MacKendrick	1
Theresa Coyne	1

Q4. Which Customers Have Above-Average Sales?

Query-

```
WITH customer_sales AS (
  SELECT c.customer_name,
      ROUND(SUM(o.sales),2) AS total_sales
  FROM orders o
  JOIN customers c ON o.customer_id = c.customer_id
  GROUP BY c.customer_id, c.customer_name
)
SELECT customer_name, total_sales
FROM customer_sales
WHERE total_sales > (SELECT AVG(total_sales) FROM customer_sales)
ORDER BY total_sales DESC;
```

Output-



Q5. What is the Highest Order Value per Customer?

Query-

```
WITH ranked_orders AS (
  SELECT c.customer_name, o.order_id,
      ROUND(SUM(o.sales),2) AS order_sales,
```

```
        RANK() OVER (
            PARTITION BY o.customer_id
            ORDER BY SUM(o.sales) DESC
        ) AS rnk
    FROM orders o
    JOIN customers c ON o.customer_id = c.customer_id
    GROUP BY c.customer_name, o.customer_id, o.order_id
)
SELECT customer_name, order_id, order_sales
FROM ranked_orders
WHERE rnk = 1
ORDER BY order_sales DESC
LIMIT 10;
```

Output-

customer_name	order_id	order_sales
Sean Miller	CA-2014-145317	23661.23
Tamara Chand	CA-2016-118689	18336.74
Raymond Buch	CA-2017-140151	14052.48
Tom Ashbrook	CA-2017-127180	13716.46
Becky Martin	CA-2014-139892	10539.90
Hunter Lopez	CA-2017-166709	10499.97
Sanjit Chand	CA-2014-116904	9900.19
Adrian Barton	CA-2016-117121	9892.74
Bill Shonely	US-2016-107440	9135.19
Sanjit Engle	CA-2016-158841	8805.04

Brief Business Insights -

1. Customer Revenue Concentration

- The top 5 customers (Sean Miller, Tamara Chand, Raymond Buch, Tom Ashbrook, and Adrian Barton) generated the highest total sales, ranging from approximately **$14K to $25K** each.
- A small group of customers contributes a significant portion of overall revenue, indicating strong revenue concentration among high-value customers.
- Retaining these customers should be a strategic priority as losing even a few could significantly impact sales.

2. Low-Value Customer Segment

- The bottom 5 customers generated less than **$25** in total sales each, with the lowest customer contributing only **$4.83**.
- These customers represent minimal revenue contribution and may require targeted engagement or promotional campaigns to increase activity.
- Several of these customers also appear among the one-time buyers, suggesting low engagement.

3. One-Time Buyers and Retention Opportunity

- **12 out of 793 customers (about 1.5%)** placed only a single order.
- One-time buyers represent a potential churn risk because they have not returned for additional purchases.
- Customer retention programs, personalized offers, or loyalty incentives could help convert these customers into repeat buyers.

4. Above-Average Customer Performance

- The average sales per customer are approximately **$2,900**.
- Customers above this average account for a large share of total revenue and represent the company's most valuable customer base.
- These customers typically place multiple orders, indicating that repeat purchasing behavior drives higher revenue.

5. High-Value Orders Drive Revenue

- The highest single order was placed by **Sean Miller (Order ID: CA-2014-145317)** with an order value of approximately **$23,661.23**.
- Other high-value orders from Tamara Chand and Raymond Buch also exceeded **$14K**, showing that a few large transactions significantly influence overall sales performance.
- Monitoring and nurturing customers who make large purchases can help maximize revenue growth.

6. Repeat Purchases Matter More Than Single Transactions

- Most top-performing customers generated their sales through multiple orders rather than a single purchase.
- This suggests that customer loyalty and repeat business contribute more to long-term revenue than occasional high-value transactions.
- Improving customer retention can therefore have a greater impact than focusing solely on new customer acquisition.

7. Customer Ranking Highlights Business Priorities

- Window-function-based ranking clearly identifies the highest-value customers and enables prioritization of marketing and customer relationship efforts.
- Ranked customer analysis helps businesses focus resources on customers who contribute the most revenue.

8. Importance of Window Functions in Business Analytics

- ROW_NUMBER() with PARTITION BY customer_id allows tracking the sequence of orders for each customer independently.
- RANK() enables customer performance comparison while properly handling ties in sales values.
- These functions are valuable for customer lifecycle analysis, ranking, retention studies, and sales performance reporting.